Work Notes: 30 Oct 2026 (Ledger checks, Excel formulas, Combining transaction files)

Oct 30, 2026 · @tanvik

Overview

Today had three threads: checking ledger data in pandas, fixing Excel formulas, and combining 60M+ rows of transaction files from two folders with different formats. The biggest lesson was memory: load only the columns and rows you need, before collecting.

#	Task	Tool	Key idea to learn
1	Ledger rows for yesterday	pandas	Half-open date range; assign `sort_values` back
2	Ledger stopped at the 27th	Polars loader	Check the data before blaming the code; Excel-saved CSVs
3	`SUMIFS` returning 0	Excel	`$` anchors, exact criteria, pivot columns
4	Number from an ID, formula shown as text, `FILTER` with OR	Excel	`REGEXEXTRACT`, Text-formatted cells, `+` means OR
5	Latest date per client, balances, export	pandas	`groupby` max, `.map()` lookups, sort by the right column, `ExcelWriter` append
6	Combining a folder of files	pandas	Collect in a list, `concat` once, `dtype=str`, memory limits
7	LazyFrames and the kernel crash	Polars	Collect only what you need; streaming; `sink_csv`
8	`transactions_loader.py`	Polars module	Required `columns`, size guard before pandas
9	Aligning ark columns	pandas	Name normalisation, clash check, `reindex`

#	Task	Tool	Key idea to learn
10	Times showing `59:48.4`	Excel / pandas	Data lost by saving a CSV in Excel

The end-of-day review comes first, then one section per task, then the error log, cheat sheet and open items.

End-of-day review (day 2 of 7)

You asked 26 questions today (15 pandas, 7 Polars, 4 Excel) and the new skill was handling data too big for memory. Three of yesterday's mistakes came back in a new form, and the PG40 Day 1 workbook is still at 0 of 40 exercises.

Solved today

- Ledger rows for a date range, latest date per client, filling balances and last trade dates with `.map()`, and exporting a result sheet.
- Three Excel formula problems: anchoring `SUMIFS`, extracting numbers from IDs, and OR logic inside `FILTER`.
- Combining 60M+ transaction rows without crashing, packaged as `transactions_loader.py`.
- Lining up the ark columns with the main transaction files.

Not confirmed yet

- Why the negative balance protection `SUMIFS` returned 0 (anchors, spelling or genuinely zero).
- Whether the ledger now reaches 29 Oct after the diagnostic.
- Which ark files lost their dates (`59:48.4`) and whether a clean copy exists.

Mistakes that came back

Pattern	Yesterday	Today
Naming a variable after a built-in	`min`, `max`	`all = []`

Pattern	Yesterday	Today
Duplicate column names	Two `Caller Ids` columns	Four `amount` columns after renaming
Latest row per key	Sorted by `CreatedAt` (card 20261029-12)	Balance sorted by `ClientID`, so `keep='first'` picked a random row
Reusing one name for different tables	—	`df` replaced by the Excel file, then `KeyError: 'ClientID'`
Excel changing data	—	`trade_id` as `2.61029E+17`, times as `59:48.4`, a formula stored as text
Loading everything at once	—	`MemoryError`, then the kernel crash on the line the previous answer had flagged

Gaps to study

- **Memory:** how big a table is (rows × columns × 8 bytes for numbers) and the difference between a lazy plan and loaded data.
- **Data types:** why a column turns into floats or mixed types, and when to use `Int64`, `dtype=str` and real dates.
- **Excel references:** relative vs absolute (`D$3`, `$A4`) when copying formulas.
- **Lookups:** `.map()` vs `merge`, and making sure the lookup side has one row per key.

Study programme check

The PG40 training database and Day 1 workbook (40 exercises) were built yesterday, and no answers have been sent yet. The 40-day Functions and Procedures notes are also in progress in another chat.

Focus for tomorrow

Do PG40 Day 1, exercises 1 to 10. Day 1 covers half-open date ranges and the session time-zone trap, which are the same ideas behind today's "yesterday" filter and the UTC timestamps.

The 7-day study plan is due on 4 Nov 2026 (day 7).

1. Ledger rows for yesterday (pandas)

Keep rows from midnight yesterday up to, but not including, midnight today. That "half-open" range (`>=` start, `<` end) catches every second of the day, including 23:59:58.

Final code

```
import pandas as pd
from ledger_loader import get_final_ledger

led = get_final_ledger(as_pandas=True)
led['CreatedAt'] = pd.to_datetime(led['CreatedAt'], format='%Y-%m-%d
%H:%M:%S')
led = led.sort_values('CreatedAt', ascending=False)      # assigned back

today = pd.Timestamp.today().normalize()      # today at 00:00:00
yesterday = today - pd.Timedelta(days=1)       # yesterday at 00:00:00

led_yesterday = led[(led['CreatedAt'] >= yesterday) & (led['CreatedAt'] <
today)]
print(yesterday.date(), '->', len(led_yesterday), 'rows')
```

How it works

- **`.normalize()`** sets the time to 00:00:00, so `today` is the very start of today.
- **`pd.Timedelta(days=1)`** is a length of time. Subtracting it moves the start back one day.
- **Two conditions need brackets and `&`.** Python's plain `and` doesn't work on whole columns; `&` compares row by row.
- **Why not `<= 23:59:59`?** Times with fractions (23:59:59.5) would slip through. `< today` has no gap.

Mistakes to avoid

- **`led.sort_values(...)` on its own line does nothing to `led`.** It returns a sorted copy and throws it away. Write `led = led.sort_values(...)`.
- **Check the data reaches yesterday.** If `led['CreatedAt'].max()` is older than yesterday, the filter correctly returns 0 rows. To get the latest day in the data instead: `led[led['CreatedAt'] >= led['CreatedAt'].max().normalize()]`.

- **Time zones.** If `CreatedAt` is stored in UTC, convert it first, or "yesterday" is 4 hours off from Dubai time:

```
led['CreatedAt'] = (led['CreatedAt'].dt.tz_localize('UTC')
                                    .dt.tz_convert('Asia/Dubai')
                                    .dt.tz_localize(None))
```

Practice

- [ ] Select the last 7 full days, not including today.
- [ ] Select only this month so far, using `today.replace(day=1)` as the start.
- [ ] Count rows per day for the last week with `led['CreatedAt'].dt.date.value_counts()`.

2. Why the combined ledger seemed to stop at the 27th (Polars loader)

The loader parses the append files correctly: `2026-10-29T23:59:53.103Z` becomes `2026-10-29 23:59:53`. So the cause is either how the result was viewed, or files not reaching the loader. Check the data before changing the code.

Step 1: is the data really missing?

```
led = get_final_ledger(as_pandas=True)
print(led['CreatedAt'].max())
```

If this prints 29 Oct, the data is there. `led.head()` was showing the first rows of the unsorted base file, because the sort wasn't assigned back (section 1).

Step 2: find where rows are lost

```
from pathlib import Path
import polars as pl
from ledger_loader import APPEND_FOLDER, scan_base_ledger, scan_append_files

files = sorted(Path(APPEND_FOLDER).glob("*.csv"))
print(len(files), "append files:", [f.name for f in files])

summary = lambda lf: lf.select(
    pl.col("CreatedAt").min().alias("first"),
    pl.col("CreatedAt").max().alias("last"),
    pl.col("CreatedAt").null_count().alias("unparsed_dates"),
    pl.len().alias("rows"),
).collect()
print(summary(scan_base_ledger()))
print(summary(scan_append_files()))
```

What you see	Cause	Fix
A day's file is missing from the list	Wrong folder, or saved as `.xlsx`	Save it as CSV in the append folder
`unparsed_dates` above 0	File opened and saved in Excel, which rewrote the dates	Re-download; never save from Excel
Appends reach 29 Oct but `head()` shows the 27th	Unsorted view	`led = led.sort_values(...)`

How it works

- **`.select(...)` with several expressions** builds one summary row. Each needs its own `.alias()`, because two columns can't both be called `CreatedAt`.
- **`null_count()`** counts dates the loader couldn't parse. The loader stores those as null instead of failing.
- **`pl.len()`** is the row count.

Two warnings from the sample data

- **`trade_id` showing `2.61029E+17`** means the file was opened in Excel. If it was saved, the full IDs are permanently rounded to 6 digits.
- **The `Z` at the end of a time means UTC.** 23:59:53Z is 03:59 the next morning in Dubai.

Practice

- [ ] Add a `rows per day` check: `group_by(pl.col('CreatedAt').dt.date()).agg(pl.len())` on the appends.
- [ ] Write a check that lists any calendar day between the first and last date with no rows.

3. Excel: `SUMIFS` returning 0

The formula summed `credit` (column I) where `ledger_type` (E) equals the header in D3 and `client_id` (C) equals A4. It returned 0 for three possible reasons, and the first is certain once the formula is copied down.

Final formula (in D4, copied down)

```
=SUMIFS('direct_ledger_report_filtered ('!$I:$I,
        'direct_ledger_report_filtered ('!$E:$E, D$3,
        'direct_ledger_report_filtered ('!$C:$C, $A4)
```

Reason 1: the header reference moved

`Sheet2!D3` has no `$`, so in row 5 it becomes `D4` (the cell above, which holds 0). Every row after the first then looks for a ledger type of 0.

Written	Copy down	Copy right	Use for
`D3`	Row changes	Column changes	Nothing fixed
`D$3`	Row fixed	Column changes	A header row
`$A4`	Row changes	Column fixed	A label column
`$I:$I`	Fixed	Fixed	A data range

Reason 2: the ledger type is spelled differently

`SUMIFS` ignores capitals but needs everything else exact. `NEGATIVE_BALANCE_PROTECTION` or a trailing space won't match. Check:

```
=COUNTIF('direct_ledger_report_filtered ('!E:E, D3)           exact matches
=COUNTIF('direct_ledger_report_filtered ('!E:E, "*NEGATIVE*")   any spelling
```

First 0 and second above 0 means the spelling differs; copy the exact value into D3, or use `"*NEGATIVE*BALANCE*"` as the criterion. `*` stands for any characters.

Reason 3: 0 is correct

The sheet covers one day, and negative balance protection entries are rare. If both `COUNTIF`s return 0, there are none.

Easier option

The target table is a pivot table already. Drag `ledger_type` into its **Columns** area with `credit` in **Values**, and you get one column per ledger type with no formula.

Practice

- [ ] Add a column for `WITHDRAW` debits per client using the same anchored pattern.
- [ ] Rebuild the table as a pivot with `ledger_type` in Columns and compare the totals.

4. Excel: numbers from IDs, formulas shown as text, and OR inside `FILTER`

Three short fixes: `REGEXEXTRACT` pulls digits out of text, a Text-formatted cell stores formulas as plain text, and `OR()` doesn't work row by row.

4a. Extract `575346` from `BR575346: ****`

Microsoft 365:

```
=REGEXEXTRACT(A2, "[0-9]+")
```

`[0-9]+` means one or more digits in a row; it returns the first such run. Put `--` in front to get a real number instead of text.

Older Excel (no regex):

```
=MID(A2, MIN(FIND({0,1,2,3,4,5,6,7,8,9}, A2&"0123456789")),
     FIND(":", A2) - MIN(FIND({0,1,2,3,4,5,6,7,8,9}, A2&"0123456789")))
```

- `FIND({0,...,9}, ...)` finds the position of each digit; `MIN` takes the first.
- Adding `"0123456789"` to the text means every digit is always found, so `FIND` never errors.
- `FIND(":", A2)` marks the end of the number; `MID` cuts out the part between.

The same in other tools: pandas `df['col'].str.extract(r'(\d+)', expand=False)`, PostgreSQL `substring(col FROM '[0-9]+')`.

4b. The formula stays as text in the cell

The cell was formatted as **Text**, so Excel stored the formula as plain text.

1. Select the cells and set **Home → Number format → General**.
2. Re-enter each formula with **F2** then **Enter**. Changing the format alone doesn't recalculate.
3. For many cells: **Ctrl+H**, replace `=` with `=`, **Replace All**.

Other causes: **Show Formulas** is turned on (toggle it with Ctrl plus the key left of 1, or **Formulas → Show Formulas**), or a space or apostrophe sits before the `=`.

4c. `UNIQUE(FILTER(..., OR(...)))` gives the wrong result

`OR()` returns one TRUE/FALSE for the whole column; `FILTER` needs one per row. Add conditions with `+` instead:

```
=UNIQUE(FILTER(dgrdReport!G:G,
               (dgrdReport!D:D="Buy") + (dgrdReport!D:D="Sell"),
               "No matches"))
```

Logic	Inside `FILTER`	Why
OR	`(A) + (B)`	1 if either is TRUE; `FILTER` keeps non-zero rows
AND	`(A) * (B)`	1 only if both are TRUE

Logic	Inside `FILTER`	Why
Several values	`ISNUMBER(MATCH(D:D, {"Buy","Sell"}, 0))`	Easy to extend the list

`"No matches"` replaces the `#CALC!` error when nothing matches. Wrap in `SORT(...)` for an ordered list, and use fixed ranges like `G2:G50000` if whole columns feel slow.

Practice

- [ ] Extract the letters only (`BR`) from `BR575346: ****`.
- [ ] Return unique symbols where side is Buy AND amount is over 1, using `*`.

5. pandas: latest date per client, filling columns with `.map()`, and exporting a sheet

The Old Portal traded-IDs sheet needed each client's latest balance and last trade date from the ledger. `groupby` finds the latest date, `.map()` copies values into existing columns, and `ExcelWriter` adds a `result` sheet to the original file.

Read a specific sheet

```
path = r"C:\Users\Administrator\Downloads\Old Portal  30 oct traded ids
(1).xlsx"
print(pd.ExcelFile(path).sheet_names)              # exact tab names
df = pd.read_excel(path, sheet_name="Sheet1")      # by name; sheet_name=0 =
first tab
```

The full working flow

```
ids = set(df['Zynixx Id'])
led_req = led[led['ClientID'].isin(ids)]

# Latest balance: sort by DATE, newest first, keep the first row per client
balance = (led_req.sort_values('CreatedAt', ascending=False)
                  .drop_duplicates(subset='ClientID', keep='first')
                  .reset_index(drop=True)[['ClientID', 'Balance']])

# Last trade date: latest BILL entry per client
bill_led = led_req[led_req['LedgerType'] == 'BILL']
```

```
last_dates = bill_led.groupby('ClientID', as_index=False)['CreatedAt'].max()

# Lookups: ClientID -> value
bal_map  = balance.drop_duplicates('ClientID').set_index('ClientID')
['Balance']
date_map = last_dates.drop_duplicates('ClientID').set_index('ClientID')
['CreatedAt']

key = df['Zynixx Id'].astype(str).str.strip()
df['Zynixx Balance'] = key.map(bal_map)
df['Zynixx Last T']  = key.map(date_map)

# Add df as a 'result' sheet in the same workbook (close it in Excel first)
with pd.ExcelWriter(path, engine='openpyxl', mode='a',
if_sheet_exists='replace') as writer:
    df.to_excel(writer, sheet_name='result', index=False)
```

How it works

- **`groupby('ClientID', as_index=False)['CreatedAt'].max()`** gives one row per client with the latest date. `as_index=False` keeps `ClientID` as a column.
- **`set_index('ClientID')['Balance']`** turns a table into a lookup: labels are IDs, values are balances.
- **`key.map(lookup)`** replaces each ID with its value. Unknown IDs, like `No New Id`, become `NaN`. The row count and order of `df` never change.
- **`.map()` vs `merge`:** `merge` adds new key and value columns you then rename and drop. `.map()` fills the column you already have.
- **`mode='a'`** keeps the other sheets; **`if_sheet_exists='replace'`** overwrites `result` on reruns instead of making `result1`.
- **To keep a whole row per client** (balance, type and date together), use the sort + `drop_duplicates` pattern; for a date column on every row, `df.groupby('ClientID')['CreatedAt'].transform('max')`.

Mistakes to avoid

- **Sorting by `ClientID` before `keep='first'`** groups each client's rows but in no date order, so the "latest" balance is random. Sort by the date.
- **`KeyError: 'ClientID'`** came from `df` being replaced by the Excel file. Check with `df.columns.tolist()`, and give each table its own name (`led`, `traded_ids`).

- **A lookup with duplicate IDs** makes `.map()` fail with `Reindexing only valid with uniquely valued Index objects`. `drop_duplicates('ClientID')` prevents it.
- **`openpyxl` rewrites the workbook**, so pivot tables, slicers and some charts can be lost. For such files, write a new workbook with both sheets instead.
- **Never run a placeholder line.** `drop_duplicates(...)` with a literal `...` raised `KeyError: Index([Ellipsis])`, because Python reads `...` as a real value.

Practice

- [ ] List the `Zynixx Id`s that got no balance, excluding `No New Id`.
- [ ] Add `Zynixx Last Deposit` using `LedgerType == 'DEPOSIT'` and the same `.map()` pattern.
- [ ] Write the result to a new file with both `Sheet1` and `result` sheets.

6. pandas: combining every file in a folder, mixed types, and the `MemoryError`

Collect each file in a list and join them once with `pd.concat`. That works until the total is bigger than memory: the transaction files hold 63.8 million rows, which pandas can't hold with every column.

The folder loop

```
import os
import pandas as pd

folder = r"C:\Users\Administrator\Videos\Office\migration\transactions"
text_cols = {'expiryDate': str, 'comment': str, 'scriptParent': str}

frames = []
for file in os.listdir(folder):
    full_path = os.path.join(folder, file)
    if file.startswith('~$'):                       # Excel lock files
        continue
    if file.lower().endswith('.csv'):
        part = pd.read_csv(full_path, dtype=text_cols)
    elif file.lower().endswith('.xlsx'):
        part = pd.read_excel(full_path)
    else:
```

```
        continue
    part['source_file'] = file
    frames.append(part)

combined = pd.concat(frames, ignore_index=True)
```

What changed from the first attempt

- **`all = []` became `frames = []`.** `all` is a built-in function, the same mistake as `min` and `max` yesterday.
- **`pd.read_csv()` needs the path,** and `.xlsx` files need `pd.read_excel()`.
- **`led = pd.DataFrame()` inside the loop was removed.** Collect in a list and `concat` once; it is far faster than adding one file at a time.
- **`os.path.join(folder, file)`**, because `os.listdir` returns names only.

`DtypeWarning: Columns (21: expiryDate) have mixed types`

`read_csv` reads big files in chunks and guesses each column's type per chunk. `expiryDate` is blank in some chunks (read as numbers) and text in others, so the column ends up mixed. Sorting it can then fail with `'<' not supported between 'str' and 'float'`.

- **Fix:** `dtype={'expiryDate': str, ...}`, then convert once: `pd.to_datetime(combined['expiryDate'], errors='coerce')`.
- **`low_memory=False`** only hides the warning; a column that really holds blanks and text is still mixed.
- **Check a column:** `combined['expiryDate'].map(type).value_counts()`.

`MemoryError: Unable to allocate 487 MiB ... shape (1, 63807297)`

One number column of 63,807,297 rows is 63.8M × 8 bytes = 487 MiB. With about 30 columns, several of them text, the table needs well over 15 GB. `pd.concat` also keeps every piece in `frames` while building the copy, so for a moment it needs about double.

Fix	Code	Saves
Read only needed columns	`pd.read_csv(path, usecols=keep_cols)`	The most memory

Fix	Code	Saves
Filter each file before keeping it	`part = part[part['clientId'].isin(ids)]`	Rows you never need
Free the pieces after joining	`del frames`	The temporary copy
Summarise per file	`part.groupby('clientId').size()` then add the results	Everything except the totals
Switch to Polars	Section 7	Reads lazily, in parallel

Practice

- [ ] Estimate a table's size before loading: rows × columns × 8 bytes, then compare with `df.memory_usage(deep=True).sum() / 1e9` on one file.
- [ ] Rewrite the loop with `usecols` and a client filter, and compare the memory used.

7. Polars: LazyFrames, the kernel crash, and appending the ark folder

A LazyFrame is a plan of what to read, not data. Nothing loads until `.collect()`, so the rule is: `select` and `filter` first, collect second, convert to pandas last.

What each call loads

Call	Reads	Memory
`pl.scan_csv(f)`	Nothing yet	None
`lf.collect_schema().names()`	Column headers	Tiny
`lf.head(5).collect()`	First rows	Tiny
`lf.select(pl.len()).collect()`	Every row, counted	Tiny result
`lf.select(cols).filter(...).collect()`	Only chosen columns and rows	Depends on the cut

Call	Reads	Memory
`lf.collect()`	Every row and column	All of it (crashed)
`.to_pandas()`	A second full copy	Doubles it
`lf.sink_csv(path)` / `lf.sink_parquet(path)`	Every row, in batches, to disk	Small, steady

Errors in order

1. **`'LazyFrame' object has no attribute 'to_pandas'`**: a plan isn't data. `.collect()` first, then `.to_pandas()`.
2. **Kernel crashed**: `lf.collect()` on 63.8M+ rows, then `.to_pandas()`. Windows stopped Python, and every variable was lost, so imports and `lf` had to be rebuilt.

The safe pattern

```
needed = (lf.select(['clientId', 'symbol', 'time'])        # columns you need
            .filter(pl.col('clientId').is_in(list(ids)))   # rows you need
            .collect(engine='streaming'))                  # older Polars:
streaming=True

print(needed.shape, round(needed.estimated_size('gb'), 2), 'GB')
needed_pd = needed.to_pandas()        # only after checking the size
```

- **`engine='streaming'`** processes the files in batches instead of all at once.
- **`group_by(...).agg(...)` before `collect`** returns only the small summary.
- **For the PostgreSQL migration**, `lf.sink_csv('transactions_combined.csv')` writes one clean file to disk for `\copy`, without holding it in memory.

Appending the ark folder (CSV and XLSX)

```
def scan_csv_file(f):
    return (pl.scan_csv(f, infer_schema_length=10_000,
schema_overrides=text_cols)
              .with_columns(pl.lit(f.name).alias('source_file')))

def read_xlsx_file(f):
    return (pl.read_excel(f, infer_schema_length=10_000)    # needs: pip
install fastexcel
              .lazy()
              .with_columns(pl.lit(f.name).alias('source_file')))

parts = [scan_csv_file(f) for f in trans_csv + ark_csv] + [read_xlsx_file(f)
for f in ark_xlsx]
lf = pl.concat(parts, how='diagonal_relaxed')
```

- **Polars can't scan Excel lazily**, so `read_excel` loads each file; `.lazy()` lets it join the plan. One `.xlsx` holds at most about 1M rows, which is fine.
- **`pl.lit(f.name)`** adds the same value to every row: the file name, for tracing.
- **`how='diagonal_relaxed'`** matches columns by name, fills missing ones with nulls, and widens types that differ. When a column is a date in Excel and text in a CSV, it becomes text.
- **Column names must match exactly.** `Ticket Id` and `ticketId` become two half-empty columns (section 9).

Practice

- [ ] Count rows per `source_file` with `group_by` and `collect(engine='streaming')`.
- [ ] Write the first 1,000 rows to a test Parquet file with `lf.head(1000).sink_parquet(...)` and reopen it with `pl.scan_parquet`.

8. `transactions_loader.py`: loading only what you need

`get_final_ledger(as_pandas=True)` never crashed because it keeps only 6 columns before collecting, so Polars reads just those from disk. `transactions_loader.py` copies that idea for the transaction files: you must name the columns, and it checks the size before converting to pandas.

The one line that keeps the ledger loader safe

```
final = final.select(list(columns))   # FINAL_COLUMNS: 6 columns, before
.collect()
```

Using the module

```
import polars as pl
from transactions_loader import list_columns, check_columns, count_rows,
get_transactions, export_transactions

list_columns()             # every column name
check_columns()            # files whose column names differ
count_rows(by_file=True)   # rows per file, without loading the data

df = get_transactions(columns=['ticketId', 'time', 'accountId', 'amount'],
                      where=pl.col('accountId').is_in(list(ids)),
                      as_pandas=True)

export_transactions(r"C:\Users\Administrator\Videos\Office\migration\all_tran
sactions.parquet")
```

Function	Does	Loads
`list_columns()`	Column names across both folders	Headers (and the Excel files)
`check_columns()`	Lists files with extra or missing columns	Headers
`count_rows(where, by_file)`	Row counts, optionally filtered or per file	Counts only
`get_transactions(columns, where, as_pandas)`	The data you name	Only those columns and rows
`export_transactions(path)`	Every row to `.parquet` or `.csv`	Batches, streamed to disk

How it protects the kernel

- **`columns` is required.** `get_transactions()` with no columns raises a `ValueError` that points you to `list_columns()`.
- **Size check before pandas.** Above `MAX_PANDAS_GB` (4 GB), it raises a clear `MemoryError` instead of crashing.
- **`_collect()` tries `engine='streaming'`**, and falls back to `streaming=True` on older Polars versions.
- **`ARK_COLUMN_MAP`** at the top renames ark columns to the transaction names; paste the mapping from section 9 there.

After editing the module, restart the kernel or run `importlib.reload(transactions_loader)`, or the notebook keeps using the old version.

Practice

- [ ] Call `count_rows(where=pl.col('action') == 'New')` and compare it with the total.
- [ ] Export one month to Parquet with `where=` on `time`, then time how long `pl.read_parquet` takes compared with the CSVs.

9. Aligning the ark columns with the main transaction files

Reduce both sets of names to lowercase letters and digits, match on that, map the few that differ by hand, and stop if two columns would get the same name. Then `reindex` to the main file's columns (minus `id`) and make the types match.

Why lowercase alone wasn't enough

Ark name	`lower()`	`norm()`	Main name
Ticket Id	ticket id	ticketid	ticketId
Total PnL	total pnl	totalpnl	totalPnL
Account	account	account	accountId (by hand)
Open Com	open com	opencom	openCommission (by hand)

Final code

```
import re

def norm(name):
    """'Ticket Id' -> 'ticketid', 'ticketId' -> 'ticketid'"""
    return re.sub(r'[^a-z0-9]', '', str(name).lower())

# Names that differ by more than spaces/capitals. Map to None to drop.
MANUAL = {'Account': 'accountId', 'Open Com': 'openCommission', 'Close Com':
'closeCommission'}

def align_columns(df, target_cols, manual=MANUAL):
    dupes = df.columns[df.columns.duplicated()].tolist()
    if dupes:
        raise ValueError(f"The file itself has duplicate column names:
{dupes}")
    lookup = {norm(c): c for c in target_cols}
    rename = {}
    for c in df.columns:
        new = manual[c] if c in manual else lookup.get(norm(c))
        if new is not None:
            rename[c] = new
    targets = pd.Series(rename)
    clashes = targets[targets.duplicated(keep=False)]
    if not clashes.empty:
        raise ValueError("These columns would get the same name:\n" +
clashes.to_string())
    return df[list(rename)].rename(columns=rename)

target_cols = [c for c in major.columns if c != 'id']
major_final = major.drop(columns='id')
ark_final = align_columns(ark, major.columns).reindex(columns=target_cols)
assert list(ark_final.columns) == list(major_final.columns)
combined = pd.concat([major_final, ark_final], ignore_index=True)
```

The four `amount` columns

The first version had no clash check. Two ark columns both became `amount` (likely `Amount` and something like `Amount ($)`, since removing symbols makes them equal). Selecting `[list(rename.values())]` then asked for `amount` twice, so each copy

appeared twice: `1.00, 37372.0, 1.00, 37372.0`. The fix selects the original columns first (`df[list(rename)]`) and stops on any clash.

One of the pair was lots (`1.00`, `4.00`); the other was close to lots × price × contract size (TESLA 1 × 374.89 × 100 ≈ 37372). Compare with `major[['amount', 'price']].head()` to decide which keeps the name.

Matching types after aligning

```
for df_ in (major_final, ark_final):
    for c in ['ticketId', 'openPosition']:
        df_[c] = pd.to_numeric(df_[c], errors='coerce').astype('Int64')   #
41672978.0 -> 41672978
    for c in ['time', 'openDate']:
        df_[c] = pd.to_datetime(df_[c], errors='coerce')

ark_final['accountId'] = ark_final['accountId'].str.replace(r':\s*\*+$', '',
regex=True)  # 'CO657078: ****' -> 'CO657078'
```

- **`reindex(columns=...)`** orders columns and adds missing ones (`fileReferenceId`, `location`) as blanks.
- **`ticketId`** shows **`41672978.0`** because blanks forced decimals. `Int64` allows blanks and keeps whole numbers.
- **Check `location`:** in one ark file it held device text like `iOS v8.1@B46DF...`, the same as `Device @ Mac Address` in another. If `major['location']` looks the same, add `'Device @ Mac Address': 'location'` to `MANUAL`.
- **Align each ark file on its own** before combining, because the files have different headers (31 columns in one, 14 in another).

Practice

- [ ] Print `pd.DataFrame({'major': major_final.dtypes, 'ark': ark_final.dtypes})` and fix every row where they differ.
- [ ] Paste the final `rename` dictionary into `ARK_COLUMN_MAP` in `transactions_loader.py` and rerun `check_columns()`.

10. Times showing `59:48.4`: data lost by saving a CSV in Excel

`59:48.4` in pandas means the date and hour are gone from the file itself. Excel displays times with fractions of a second as minutes:seconds, and saving a CSV writes what is displayed. The same rows exist with full times (`2026-09-01 00:59:48.4`) in another ark file, so a clean copy is likely.

1. Find the damaged files

```
short = ark_clean['time'].astype(str).str.fullmatch(r'\d{1,2}:\d{2}(\.\d+)?')
# mm:ss or mm:ss.f
print(short.sum(), 'rows with only mm:ss')
ark_clean.loc[short, 'source_file'].value_counts()
```

`reindex` drops `source_file`, so run this on `ark_clean`, which still has it. `fullmatch` requires the whole value to fit the pattern, so full dates don't count.

2. Look for a clean copy and duplicates

```
print(ark_clean.duplicated(subset='ticketId').sum(), 'duplicate ticketIds')
dup_ids = ark_clean.loc[short, 'ticketId']
ark_clean[ark_clean['ticketId'].isin(dup_ids) & ~short][['ticketId', 'time',
'source_file']].head()
```

If the same rows are in a `.csv` and an `.xlsx`, every row is loaded twice. Removing the damaged files fixes the times and the duplicates together.

3. No clean copy: blank the time, don't guess

```
t = ark_final['time'].astype(str).str.strip()
short = t.str.fullmatch(r'\d{1,2}:\d{2}(\.\d+)?')
ark_final['time'] = pd.to_datetime(t.where(~short), format='ISO8601',
errors='coerce')
```

- **`t.where(~short)`** keeps good values and turns damaged ones into blanks.
- **`format='ISO8601'`** reads both `2026-09-01 00:59:38` and `2026-09-01 00:59:48.400000`.
- **`openDate`** (`28:14.3`) has the same damage; run the same steps on it.

Excel damage seen today

What you saw	What Excel did	Recoverable?
`59:48.4` in `time`	Saved the minutes:seconds display	No; use the original file
`2.61029E+17` in `trade_id`	Rounded an 18-digit ID to 6 digits	No, if saved
`=REGEXEXTRACT(...)` shown as text	Cell formatted as Text	Yes: General, then F2 + Enter

Stop it happening again

- **Never save a CSV after opening it in Excel.** Close without saving.
- **Look at CSVs in pandas:** `pd.read_csv(path, nrows=20)`.
- **If you must edit in Excel,** format time columns as `yyyy-mm-dd hh:mm:ss.0` and long IDs as Text before saving.

Practice

- [ ] Write a check that runs on every new file and stops if any `time` value fails `fullmatch(r'\d{4}-\d{2}-\d{2}.*')`.

Error log

Most of today's errors came from data being bigger, messier or differently named than the code assumed. Each row links to the section with the full fix.

Error or symptom	Tool	Cause	Fix	Section
`head()` showed only the 27th	pandas	`led.sort_values(...)` not assigned back	`led = led.sort_values(...)`	1, 2
`SUMIFS` returned 0	Excel	`D3` moved when copied; possible spelling mismatch	`D$3`, `$A4`; check with `COUNTIF`	3
Formula displayed as text	Excel	Cell formatted as Text	General, then F2 + Enter	4b

Error or symptom	Tool	Cause	Fix	Section
`FILTER` wrong with `OR()`	Excel	`OR` returns one value for the whole column	`(A) + (B)`	4c
`KeyError: 'ClientID'`	pandas	`df` replaced by the Excel file	Check `df.columns`; one name per table	5
Wrong latest balance	pandas	Sorted by `ClientID`, not `CreatedAt`	Sort by the date before `keep='first'`	5
`KeyError: Index([Ellipsis])`	pandas	Ran a placeholder `drop_duplicates(...)`	Run only real code	5
Loop read nothing / failed	pandas	`pd.read_csv()` with no path; `all` shadowed a built-in	`os.path.join`; name it `frames`	6
`DtypeWarning: mixed types`	pandas	Blanks and text in one column, guessed per chunk	`dtype={'expiryDate': str, ...}`	6
`MemoryError: 487 MiB`	pandas	63.8M rows × every column	`usecols`, filter early, or Polars	6
`'LazyFrame' object has no attribute 'to_pandas'`	Polars	A plan isn't data	`.collect()` first	7
Kernel crashed	Polars	`lf.collect()` on everything, then `.to_pandas()`	`select` + `filter` + streaming	7
Four `amount` columns	pandas	Two ark columns renamed to one name	`align_columns` with a clash check	9
`ticketId` as `41672978.0`	pandas	Blanks forced decimals	`.astype('Int64')`	9
`time` as `59:48.4`	Excel	CSV saved from Excel	Use the original file; blank damaged values	10
`trade_id` as `2.61029E+17`	Excel	Excel's number display (lost if saved)	Keep original downloads untouched	2, 10

Cheat sheet: new today

Only functions that weren't in yesterday's cheat sheet.

pandas

Function	What it does	Example
`pd.Timestamp.today().normalize()`	Today at midnight	Start of a date range
`pd.Timedelta(days=1)`	A length of time	`today - pd.Timedelta(days=1)`
`.dt.tz_localize('UTC').dt.tz_convert('Asia/Dubai')`	Change a time zone	UTC ledger times
`pd.ExcelFile(path).sheet_names`	List a workbook's tabs	Before `sheet_name=`
`pd.read_excel(path, sheet_name='Sheet1')`	Read one tab	Traded IDs file
`groupby(key, as_index=False)[col].max()`	Latest value per key	Last trade date
`groupby(key)[col].transform('max')`	Group value on every row	Latest date beside each row
`set_index(key)[col]` + `series.map(lookup)`	Look values up by key	`Zynixx Balance`
`pd.ExcelWriter(path, mode='a', if_sheet_exists='replace')`	Add or replace a sheet	`result` sheet
`pd.read_csv(path, usecols=..., dtype=...)`	Read only some columns, fixed types	Big files
`df.memory_usage(deep=True).sum()`	Real memory used	Size checks

Function	What it does	Example
`df.reindex(columns=cols)`	Set column order, add missing as blank	Aligning ark data
`series.str.fullmatch(pattern)`	Whole value matches a regex	Finding `59:48.4`
`series.where(cond)`	Keep values where True, blank elsewhere	Blanking damaged times
`pd.set_option('display.max_columns', None)`	Show every column	Wide tables

Polars

Function	What it does	Example
`pl.scan_csv(f, schema_overrides=...)`	Lazy read with fixed types	Transaction CSVs
`pl.read_excel(f).lazy()`	Read Excel, then join a lazy plan	Ark `.xlsx` files
`pl.concat(parts, how='diagonal_relaxed')`	Combine; match by name, widen types	All folders
`lf.collect_schema().names()`	Column names without reading data	Exploring
`lf.select(pl.len()).collect()`	Row count	Size check
`lf.collect(engine='streaming')`	Run the plan in batches	Big filters

Function	What it does	Example
`df.estimated_size('gb')`	Size of a loaded table	Before `.to_pandas()`
`lf.sink_csv(path)` / `lf.sink_parquet(path)`	Write everything to disk in batches	Migration export
`pl.lit(value).alias(name)`	Same value on every row	`source_file`

Excel

Function	What it does	Example
`D$3`, `$A4`, `$I:$I`	Fix the row, the column, or both	`SUMIFS` copied down
`REGEXEXTRACT(A2, "[0-9]+")`	First run of digits	`575346`
`MIN(FIND({0,...,9}, A2&"0123456789"))`	Position of the first digit	Older Excel
`FILTER(range, (A)+(B), "No matches")`	OR inside `FILTER`	Buy or Sell
`ISNUMBER(MATCH(range, {"Buy","Sell"}, 0))`	Value is in a list	Longer OR lists
`COUNTIF(range, "*NEGATIVE*")`	Count with a wildcard	Checking spellings

Open items

Three checks from today are still waiting on a result, and yesterday's tagging decisions are still open.

- [ ] Run the two `COUNTIF` checks to find why the negative balance protection column is 0.
- [ ] Confirm `led['CreatedAt'].max()` reaches 29 Oct after the loader diagnostic.
- [ ] Find which ark files show `59:48.4` and whether a clean copy of those rows exists; remove the damaged files.
- [ ] Check for duplicate `ticketId`s between the ark `.csv` and `.xlsx` files.

- [ ] Decide which ark column is the real `amount`, and whether `Device @ Mac Address` maps to `location`.
- [ ] Compare the `accountId` format in both tables (`CO657078` vs `CO657078: ****`).
- [ ] Paste the final column mapping into `ARK_COLUMN_MAP` in `transactions_loader.py`.
- [ ] From 29 Oct: confirm with the manager whether to tag all 205 automatic traders or only High and Medium, and whether `Call Disconnected` is unreachable.
- [ ] Start PG40 Day 1, exercises 1 to 10.